

requirement_agent整体流程

这个项目本质上是一套 “AI 辅助的需求管理流水线”：



最重要的设计原则是：AI 只生成建议，不直接修改正式需求。正式版本只能由人工审核后产生。

一、整体架构

服务	功能
React+Vite	管理后台
FastAPI	接收请求、查询数据、执行审核操作
Celery Worker	解析附件、执行AI workflow、构建索引
Redis	Celery 消息队列
PostgreSQL + pgvector	业务数据、版本、审计记录和向量
MinIO	保存 PDF、Word、图片原文件
LLM/Embedding 服务	结构化提取、冲突分析和向量生成

二、用户提交需求

用户主要从前端 “提出需求” 页面提交：

- 只输入文字；
- Dock, PDF；
- PNG/JPEG 截图。

前端根据是否有附件选择接口：

纯文本 → POST /api/v1/ingestions

有附件 → POST /api/v1/files

具体提交逻辑位于 apps/web/src/pages/ConversationPage.tsx:129。

每次提交都会携带 Idempotency-Key。后端根据渠道事件 ID 和内容指纹判断：

- Key 和内容都相同：返回原记录，不重复创建；
- Key 相同但内容不同：返回幂等冲突；
- 新 Key：创建新的 SourceRecord。

接入层通过 Connector 将不同渠道统一为同一种内部数据结构。当前实现有：

- WebFormConnector
- DocumentConnector
- ImageConnector
- 枚举中还预留了飞书渠道

统一接入过程在 src/requirement_agent/application/ingestion/service.py:54。

三、原始数据持久化

后端会先创建不可覆盖的原始记录：

SourceRecord

- └—— 原始文本
- └—— 提交人
- └—— 渠道
- └—— 外部事件 ID
- └—— 内容指纹
- └—— 处理状态

如果有附件，还会创建 SourceAttachment：

SourceAttachment

- └—— 文件名和 MIME
- └—— 文件哈希
- └—— MinIO 存储路径
- └—— 解析文本
- └—— OCR 文本
- └—— 解析状态

流程是：

1. 校验文件扩展名、MIME、文件签名和大小；
2. 将 SourceRecord 和附件元数据写入 PostgreSQL；
3. 将附件二进制写入 MinIO；
4. 写入审计日志；
5. 向 Celery 投递后台任务。

对应代码见 `src/requirement_agent/application/ingestion/service.py:67`

这里有一个很好的可靠性设计：原始记录先保存，再启动异步处理。即使 AI 或文件解析失败，用户提交的原始证据仍然存在。

四、Worker 解析附件

API 返回 202 Accepted 后，真正耗时的工作交给 Celery Worker。

第一步是 `requirement_agent.sources.parse` 任务：

1. 把来源状态改成 parsing；
2. 从 MinIO 下载附件；
3. 根据文件类型选择解析器；
4. PDF/DOCX 提取正文；
5. 图片执行 OCR；
6. 把结果写回附件记录；
7. 投递后续 AI 分析任务。

任务定义在 `src/requirement_agent/application/ingestion/tasks.py:42`。

附件解析失败会：

- 将附件标记为 failed；
- 将来源标记为 `parse_failed`；
- 写审计日志；
- 最多自动重试三次。

纯文本输入虽然没有附件，也会走同一个任务链，只是解析阶段没有实际附件需要处理。

五、AI 分析 workflow

解析完成后，Worker 执行 `RequirementAnalysisWorkflow`。

LangGraph 定义了四个固定节点：

`extract` → `retrieve` → `analyze` → `finalize`

图的定义位于 `src/requirement_agent/workflows/requirement_analysis.py:126`。

5.1extract：结构化提取

首先将下面内容合并：

用户填写的 raw_text

- 文档解析正文 parsed_text
- 图片 OCR 文本 ocr_text

然后交给聊天模型，提取成标准结构，例如：

- 需求摘要；
- 需求描述；
- 功能模块；
- 验收条件；
- 业务实体；
- 待澄清问题。

模型必须输出符合 Pydantic Schema 的 JSON。输出格式不正确时，系统会把具体校验错误反馈给模型并自动纠正。

此外还会加载已有模块名，引导模型复用已有模块，避免出现“用户管理”“用户管理系统”“用户模块”这类近义模块泛滥。

5.2retrieve：检索历史需求

系统不会让模型凭空判断冲突，而是先从正式需求库检索候选。查询内容使用提取后的：

需求摘要 + 标准化需求描述

检索综合三类信号：

- 关键词全文检索
- pgvector 向量相似度
- 功能模块等业务字段匹配

此外，还会单独进行原始文本精确重复匹配。精确重复候选会被放在普通语义候选前面。

对应代码见 src/requirement_agent/workflows/requirement_analysis.py:224。

5.3analyze：冲突和风险分析

聊天模型收到：

- 当前需求的结构化结果；
- 后端检索出的历史需求候选。

然后判断当前需求属于：

- 无冲突；
- 相关；

- 重复；
- 矛盾；
- 替代旧需求；
- 依赖其他需求；
- 信息不足。

同时生成：

- 风险项；
- 澄清问题；
- 建议执行的功能变更；
- add / modify / delete / restore 操作。

系统会校验模型引用的 requirement_key，确保它只能引用检索器实际返回的候选，防止模型虚构历史需求。

5.4finalize：转入人工审核

分析完成后系统创建 ReviewTask，保存三份快照：

extraction_snapshot AI 结构化提取结果

candidate_snapshot 历史需求候选

analysis_snapshot 冲突、风险和操作建议

然后把来源状态设置成 pending_review。创建过程是幂等的，重复执行任务不会为同一分析结果反复创建审核任务。

对应代码见 src/requirement_agent/workflows/requirement_analysis.py:528。

六、人工审核

审核人员可以：

- 批准并新建正式需求；
- 批准并合并到已有需求；
- 修改 AI 给出的标题、模块、描述和验收标准；
- 驳回，不保留该需求；
- 退回补充；
- 要求重新分析。

批准时有两种决策：

create → 创建新的 Requirement，版本号从 1 开始

merge → 修改已有 Requirement，版本号递增

七、正式需求版本化

批准之后，VersionCommitService 在一个数据库事务中完成：

1. 锁定审核任务；
2. 新建需求或锁定目标需求；
3. 在当前功能列表上应用操作；
4. 创建完整的 RequirementVersion 快照；
5. 保存每个 RequirementFeature；
6. 保存功能来源 FeatureLineage；
7. 更新需求的 current_version_id；
8. 将审核任务标记为 approved；
9. 将来源标记为 versioned；
10. 写入审核及版本审计日志。

核心代码在 src/requirement_agent/application/versions/service.py:77。

八、版本建立检索索引

版本提交成功后，API 再向 Celery 投递索引任务。

Worker 会：

1. 读取新版本及全部有效功能；
2. 生成一份需求级文档；
3. 为每个功能生成一份功能级文档；
4. 调用 Embedding 模型；
5. 写入 RequirementEmbedding；
6. 后续检索只使用需求当前版本的投影。

对应实现是 src/requirement_agent/application/versions/indexing.py:20。

因此整个系统形成闭环：

新需求

→ 检索已有正式版本

→ 人工确认

→ 产生新正式版本

→ 为新版本建立索引

→ 成为下一条需求的检索候选

九、整体流程介绍

这个项目是一个多渠道需求管理 Agent，主要解决产品需求来自网页文本、PDF、Word、截图、飞书等不同渠道，难以统一整理、难判断与历史需求是否重复或冲突、以及需求版本难追溯的问题。

整体流程从用户提交需求开始。前端使用 React 负责页面交互，比如聊天页面提交需求，提交附件或者图片等、TypeScript 约束前端数据类型，确保接口数据用对。 Ant Design 提供现成的按钮、表格、表单、弹窗等 UI 组件，实现对话式输入页面，用户可以输

入文本或上传 PDF、Word、图片。点击发送后，前端调用 FastAPI 接口，FastAPI 就是前端 React 页面和后端 Agent 系统之间的接口层：纯文本调用 `/api/v1/ingestions`，带附件则调用 `/api/v1/files`。

FastAPI 接收请求后，会先校验参数、文件大小、MIME 类型（.txt;.png;.pdf等）和文件真实签名，然后创建一条原始需求记录 SourceRecord（记录输入渠道、提交人、原始文本、提交时间、处理状态）

再通过 Connector 将网页、文档、图片等不同来源统一转换为内部的原始需求对象。之后由 `IngestionService` 将原始文本、提交人、来源渠道和处理状态写入 PostgreSQL；附件元数据也写入 PostgreSQL，实际文件保存到 MinIO。

为了防止用户重复点击或网络重试产生重复需求，接口要求携带 `Idempotency-Key`。系统会结合渠道、幂等 Key 和请求内容指纹判断：相同 Key、相同内容直接返回已有记录；相同 Key、不同内容则拒绝请求。

用户上传对话文本/附件，原始文件会上传到MinIO，数据库创建SourceRecord 和 SourceAttachment，然后投递 PARSE_SOURCE_TASK到Redis，FastAPI会立即返回。这里上传接口不会一直等待PDF解析、图片OCR、Agent 分析，而是通过 Celery 将任务投递到 Redis。Redis 相当于任务队列，Celery Worker 从Redis队列中取任务执行。

这样用户可以很快看到“需求已保存，AI 正在分析”，而 PDF 解析、OCR、调用大模型和向量检索都在后台完成。

Celery是一个异步任务队列框架，用于把耗时任务从 FastAPI 主接口中拆出去后台执行。上传大文件、OCR、LLM调用可能花费较长时间，这样也不会卡住用户请求。并且这里Celery Worker配置了concurrency: 6 (prefork)，也就是一个Worker容器内部有6个并发执行槽位。支持多会话任务同时执行。

Worker启动时，会调用register_tasks(celery_app)，用于向Celery登记四种任务。这些任务可由Celery 可投递、Worker 消费

任务名	对应函数	作用
PARSE_SOURCE_TASK	parse_source_task()	解析 PDF、Word、图片
ANALYZE_SOURCE_TASK	analyze_source_task()	启动 Agent 工作流
INDEX_VERSION_TASK	index_version_task()	正式版本生成向量
FEISHU_EVENT_TASK	process_feishu_event_task()	接收飞书消息/附件

这里Celery的异步体现在：**接口不等待耗时分析完成，而是先把任务交给后台 Worker，接口可以立即返回：已接收，然后更新状态为pending/processing。Celery Worker 在后台解析 PDF、OCR、调用 LLM、检索历史需求**

Celery Worker 收到 `PARSE_SOURCE_TASK(101)` 后

Worker 首先解析附件：PDF 使用 PyMuPDF，Word 使用 python-docx，图片使用 PaddleOCR。解析后的文字会与用户原始文本合并，形成统一的 Agent 输入。

前面输入的文本和附件都解析完成后，也就是parse_source_attachments()正常结束，会自动投递 `ANALYZE_SOURCE_TASK(101)` 到Redis队列，worker后续消费。这样也能保证不会出现文本/附件还没解析完，Agent就开始分析的情况。

Agent 部分使用 LangGraph 编排成受控的工作流，主要包括四个节点extract、retrieve、analyze、finalize。

第一步是需求提取_extract()，LLM 将原始文本转换为结构化需求，包括需求摘要、标准描述、功能模块、验收条件、风险和待澄清问题。这里会调用

`StructuredLLM`，按 `RequirementExtraction` Schema 输出结构化 JSON。主要流程如下：模型输出原始JSON数据，由 Pydantic进行校验，然后回进行**业务校验**，不合法的会要求模型自动修正，然后还会指定最多重试次数，每次调用大模型都会写入

AnalysisResult记录。所以后续节点拿到的都是经过验证的对象，而不是随意的模型文本。

第二步是 RAG 检索_retrieve(), 前面需求提取完成后会将 requirement_summary和requirement_description进行拼接，然后调用Embedding模型生成查询向量query，这里Embedding模型使用的是text-embedding-v4，

再从**审核通过的历史正式需求**中进行关键词、向量语义和业务模块三路**混合检索**。这里混合检索的得分是

```
Code block
1  score=0.4×keyword+0.4×vector+0.2×business
```

最后按综合得分降序排列，取前limit个结果。

关键词匹配得分：使用 PostgreSQL 的全文搜索功能，计算查询文本与需求标题和内容的匹配度。

会先将**历史正式需求标题和需求正文**转换为可全文检索的词条集合。

然后将本次查询向量query 转化为待匹配的词条条件。

计算查询文本和正式需求标题/正文的匹配程度。根据命中的词、词出现次数和词之间的紧密程度计算分数。

目前分词策略主要使用的是PostgreSQL内置的简单分词（不会做词型还原和同义词扩展），目前该策略更接近基础词条匹配，后续可以优化。

向量匹配得分：计算查询向量和需求向量的余弦相似度

用于判断两条需求的**语义是否相近**，不只看关键词是否完全一样。

例如

```
Code block
1  新需求：音乐播放器增加 0.5 倍速
2  历史需求：允许用户调整音频播放速率
```

两句话关键词不完全相同，但语义都是“调整播放速度”，所以两个向量在高维空间中距离较近， vector_score 向量匹配得分会较高

业务匹配得分：是一个**规则型加分项**：判断“当前新需求”和“历史需求”是否属于同一个功能模块。

比如当前需求提取后得到

```
Code block
1  query_modules = ["音乐播放器", "播放控制"]
```

历史需求向量记录的模块是：

```
Code block
1  RequirementEmbedding.module = "播放控制"
```


那么这个历史需求的业务匹配得分就是1，否则为0

第三步是冲突和风险分析 `_analyze()`，LLM 基于当前需求、RAG检索召回到的候选需求，预设的冲突分析prompt，判断它是新增、关联、重复还是冲突，并给出风险和功能增删改建议。最后一步是生成待人工审核任务。模型输出的 `ConflictAnalysis`，包括：需求是否重复、是否关联或依赖、冲突项、风险项、待确认的问题、建议的操作(new, add, modify, delete)

这里有两个安全设计：第一，大模型的输出必须通过 Pydantic 严格校验，字段类型、长度和增删改操作规则不合法时会自动纠错重试；第二，模型只能引用 RAG 检索到的候选需求，后端会校验它不能编造不存在的需求编号。

第四步人工审核 `_finalize()`，只创建待审核结果，不直接改正式需求。

把 Agent 的分析结果交给人工审核环节，体现了 Human-in-the-loop（人工在环）设计。

Agent 分析结束后，不会直接把结果写成正式需求，而是保存提取结果、候选需求、冲突与风险快照，然后创建 `ReviewTask`，更新需求状态为 `pending_review`。

这里是项目中比较重要的地方，LLM可以分析需求，提出建议，但是只有人工才能正式确定该需求是否符合条件入库。这样即使 LLM分析错误，也不会污染需求数据库。同时系统会记录 `AnalysisResult`、原始来源、调用模型、Prompt版本、调用时间、原始输出和相关错误信息方便后续回溯排查。

审核人员在前端查看 Agent 的提取结果、候选历史需求、冲突风险和修改建议后，可以批准、驳回或退回重新分析。

只有审核通过后，系统才会生成新的正式需求版本，并把该版本及其中的功能点向量化，写入 PostgreSQL 的 pgvector。

版本提交完成后，Celery投递 `INDEX_VERSION_TASK`，Celery Worker 后续会执行 `index_requirement_version`。

`index_requirement_version()` 不会把原始 PDF 全文直接切块入库。它使用审核后的、规范化的需求版本，构造：需求级文档 + 功能级文档。

需求级：用于理解整个需求，适合召回“整体相似”的历史需求

Code block

- 1 音乐播放器播放倍速设置
- 2 播放倍速：支持 0.5、1.0、1.5、2.0 倍速
- 3 歌词同步：倍速切换后同步歌词进度

功能级：用于命中具体功能点，适合召回细粒度需求

每个 `active` 功能点各生成一条，例如：

Code block

- 1 播放倍速
- 2 支持 0.5、1.0、1.5、2.0 倍速
- 3 默认 1.0 倍；切换即时生效

Code block

- 1 歌词同步
- 2 倍速切换后同步歌词进度
- 3 歌词与播放进度偏差不超过.....

一个需求通常会产生：

1 条需求概要向量 + N 条有效功能点向量，被删除或非活跃的功能点不会生成长期检索向量。

每条 RequirementEmbedding 除了 embedding，还保存了检索和溯源所需的元数据：

```
Code block
1 requirement_key、version_number
2 feature_key、module
3 标题、状态、功能模块
4 完整功能点列表
5 content：本次真正向量化的文本
6 sources：来源需求、提交人、渠道、证据文本
7 embedding_model：由哪个模型生成
```

插入 RequirementEmbedding.embedding 后，PostgreSQL 的 pgvector 索引会维护 HNSW 图。它负责在大量正式需求向量中，快速找出与新需求查询向量接近的候选

这里，HNSW 索引的是**已审核正式需求的规范化版本内容**，不是 PDF 原文，也不是会话记忆。

因此，这个项目形成了一个完整闭环：用户输入需求，系统异步解析和分析，Agent 给出有历史依据的建议，人工审核后生成正式版本，再沉淀为下一次 RAG 可检索的知识库。整个设计的核心是让 AI 提升需求分析效率，但不让 AI 直接修改正式业务数据。

十、代码流程

顺序	要看代码	重点看什么
1	apps/api/ main.py	FastAPI 启动入口，路由如何注册。
2	src/requirement_agent/api/routes/ sources.py	POST /api/v1/ingestions：前端提交文本、PDF、Word、图片后首先进入这里。
3	api/schemas/ sources.py 、api/ dependencies.py	请求参数如何校验；如何创建 IngestionService。
4	application/ingestion/ service.py	创建 SourceRecord、保存原始文本和附件信息、设置处理状态、触发后续任务。
5	connectors/ web_form.py 、connectors/ document.py 、connectors/ image.py	不同渠道的数据如何统一转换成内部的需求输入。
6	domain/sources/ entities.py	看 SourceRecord、AttachmentInput 等领域对象，理解系统如何描述“一次原始需求”。
7	infrastructure/storage/、infrastructure/database/models/	文件存 MinIO 的逻辑；SourceRecord、附件、需求版本、审核任务等数据库模型。
8	application/ingestion/ dispatcher.py	服务层怎样把“解析/分析任务”投递到队列。
9	infrastructure/queue/ celery.py 、apps/worker/ celery_app.py	Celery 和 Redis 的配置；Worker 如何启动并消费任务。
10	application/ingestion/ tasks.py	最重要的衔接层。重点看 parse_source_attachments()、parse_source_task()、analyze_source_task()、analyze_source()。最终会调用 Agent 工作流。
11	workflows/ requirement_analysis.py	**Agent 核心代码。**看 AnalysisState、RequirementAnalysisWorkflow 和图结构。
12	同文件的 _extract()	Agent 的第一个节点：读取当前需求及附件解析文本，提取标准化需求。重点看 _source_content()。
13	ai/prompts/ templates.py	EXTRACTION_SYSTEM_PROMPT：模型被要求提取哪些字段、如何限制中文输出、缺失信息如何提问。
14	ai/schemas/ analysis.py	看 RequirementExtraction、RequirementEntities 等 Pydantic 模型。它决定模型最终必须返回什么 JSON。
15	ai/ structured.py	StructuredLLM：调用模型、校验 JSON、失败时自动纠错重试的实现。
16	ai/llm/ base.py 、ai/llm/ factory.py 、ai/llm/ openai_compatible.py	LLM 适配层。重点是 complete()、_post()：项目如何兼容 OpenAI 风格接口。
17	回到 workflows/ requirement_analysis.py 的 _retrieve()	Agent 第二个节点：拿当前需求去找历史正式需求。你前面问的“记忆”主要就在这里生效。
18	ai/retrieval/ hybrid.py	HybridRetriever、RetrievalWeights、SearchFilters。看关键词、向量、业务模块三种分数怎样融合。
19	ai/modules/	功能模块名称如何归一化，例如避免“播放控制模块”和“播放模块”被当成两个完全不同模块。
20	回到工作流的 _analyze()	Agent 第三个节点：把“当前提取结果 + 检索到的历史候选需求”交给模型，判断重复、关联、冲突和风险。
21	ai/prompts/ templates.py 中的 CONFLICT_SYSTEM_PROMPT	冲突分析时模型的角色、输出约束和判断规则。
22	ai/schemas/ analysis.py	看 ConflictAnalysis、ProposedOperation、风险/冲突相关模型，理解截图中 related、dependency、风险提示从哪里来。
23	workflows/ requirement_analysis.py 中的 _enforce_exact_duplicates()、_validate_candidate_scope()	这是 Agent 的规则兜底：防止模型乱引用未检索到的需求，也能把完全重复的需求强制标记为重复。
24	工作流的 _finalize()	Agent 最后一个节点：不直接修改正式需求，而是创建 ReviewTask，状态变为 pending_review。
25	api/routes/ reviews.py 、审核相关 service	人工查看分析结果、确认保留/修改/拒绝的接口。
26	VersionCommitService 及版本相关 domain/service	审核通过后创建正式需求版本、维护版本血缘和当前版本、写审计日志。
27	向量写入相关 repository/model	正式需求生成或保存 Embedding 后，才会成为后续 _retrieve() 可检索的长期“记忆”。

agent相关的主流程

workflows/requirement_analysis.py

application/ingestion/tasks.py

ai/retrieval/hybrid.py

ai/structured.py

ai/schemas/analysis.py